SPI Interview Cheat Sheet — Q1–12

Nov 5, 2026 · @Dmytro

Fundamentals (Q1–Q7)

Say each answer out loud in about 15–20 seconds. The bold line is the core you must not forget.

Q1 — Why no addresses and no ACK?

CS replaces addressing; software checks replace ACK. Each slave has its own CS line, so selection is physical. There is no hardware ACK. You verify the link by reading a register with a known value (MCP2515 CANSTAT = 0x80 after RESET) or by polling a status register (M95320 RDSR/WIP). A missing slave fails silently: MISO just reads 0xFF or 0x00.

Q2 — What does "every transfer is an exchange" mean?

Two 8-bit shift registers wired in a ring; 8 SCK pulses swap their contents. Each pulse shifts one bit out of the master and one bit out of the slave at the same moment. The master makes the clock, so to receive N bytes it must send N bytes (dummy 0x00/0xFF). On the F411, writing DR is the only thing that starts SCK. Bytes received during the command and address phases are meaningless.

Q3 — CPOL and CPHA; mode 0 edges?

Mode 0: SCK idles low, sample on rising, change on falling. CPOL = idle level of SCK. CPHA = sample on the first edge (0) or the second edge (1); rising or falling depends on CPOL. Rule: data changes on one edge and is sampled on the opposite one, giving each bit half a period to settle. In CPHA=0 modes the first bit appears when CS goes low.

Mode	CPOL	CPHA	Idle	Sample	Change
0	0	0	low	rising	falling
1	0	1	low	falling	rising
2	1	0	high	falling	rising
3	1	1	high	rising	falling

Shortcut: CPOL = CPHA → sample on rising. CPOL ≠ CPHA → sample on falling.

Q4 — What if master and slave use different modes?

What matters is whether both sample on the same physical edge. If they don't, one side reads each bit exactly while the other is changing it. Symptoms: bytes shifted by one bit (CANSTAT 0x80 reads as 0x40), commands ignored, errors that come and go with clock speed or temperature. Modes 0 and 3 often work together (both sample on rising), as do 1 and 2 (both on falling). Check by reading a register with a known reset value.

Q5 — SPI vs I2C vs UART; when SPI?

	SPI	I2C	UART
Wires (+GND)	3 + 1 CS per slave	2	2
Typical speed	1–50 MHz	100 kHz / 400 kHz / 1 MHz	9600–115200 baud
Duplex	full	half	full
Clocking	synchronous, master drives SCK	synchronous, master drives SCL (slave can stretch it)	asynchronous, agreed baud rate
Many devices	1 CS per slave, one master	addresses on a shared bus, multi-master possible	point-to-point peers (LIN: 1 master + up to 16 slaves)

Pick SPI when throughput or latency matters: CAN controllers, flash and EEPROM, displays, fast ADCs. Costs: one CS pin per slave, no ACK, no standard command set. Example: one 14-byte MCP2515 frame takes about 22 µs at 5 MHz SPI, versus about 350 µs over I2C at 400 kHz.

Q6 — Three slaves on one master; daisy-chaining?

Star: shared SCK/MOSI/MISO + one CS each = 3 + N pins (6 for three slaves). Every deselected slave must release MISO (high impedance), or two drivers fight on the line. Daisy-chain: one shared CS, each slave's DOUT feeds the next one's DIN, last DOUT goes to MISO. The chain acts as one long shift register, and every slave latches its byte when CS rises. Always 4 pins, but every transfer goes through the whole chain. Only some devices support it (74HC595, MAX7219); command/response devices like the MCP2515 and M95320 don't.

Q7 — Why is SPI faster than I2C?

SPI lines are push-pull (both edges driven); I2C is open-drain (rising edge only through a pull-up). I2C lines rise as the pull-up charges the bus capacitance: τ = R × C, e.g. 4.7 kΩ × 200 pF ≈ 0.94 µs, against a 300 ns max rise time in Fast mode. I2C also spends 9 clocks per byte (ACK) plus start, stop and address. SPI spends 8 clocks per byte and moves data both ways.

STM32F411 register level (Q8–Q12)

Register	Bits that matter
RCC	SPI1 → APB2ENR bit 12 · SPI2/SPI3 → APB1ENR bits 14/15
CR1	CPHA[0], CPOL[1], MSTR[2], BR[5:3], SPE[6], LSBFIRST[7], SSI[8], SSM[9], DFF[11]
SR	RXNE[0], TXE[1], MODF[5], OVR[6], BSY[7]

Per byte: wait TXE → write DR → wait RXNE → read DR. End of transfer: last RXNE → TXE=1 → BSY=0 → raise CS.

Q8 — TXE, RXNE, BSY; which before raising CS?

Wait for BSY=0 before raising CS. TXE=1: the buffer can take the next byte; the previous byte may still be shifting (not "sent", same trap as USART TXE vs TC). RXNE=1: a received byte is ready in DR. BSY=0: the shift register has finished and SCK has stopped. Raising CS early cuts off the last byte: the MCP2515 aborts the command, and the M95320 silently skips the write (CS must rise on a byte boundary). Pace bytes with TXE/RXNE, never with BSY.

Q9 — Master writes but never reads DR?

Writes keep working; the failure appears later, at the first read. RXNE stays 1 and OVR gets set, so newly received bytes are thrown away. At the next read, the RXNE wait passes instantly and returns a stale byte from the earlier writes. It looks like a slave or wiring fault. Fix: read DR after every byte. Clear OVR by reading DR, then SR.

Q10 — MODF; why a fresh master falls back to slave?

After reset NSS is a hardware input (SSM=0, SSOE=0); NSS low → MODF → MSTR and SPE cleared. A floating or unconfigured NSS pin (PA4 for SPI1) triggers it as soon as you enable. The driver then hangs waiting for RXNE, because a disabled slave never clocks. Fix: SSM=1 **and** SSI=1 (SSI resets to 0, and SSM alone still faults). Drive each CS from a GPIO; hardware NSS output holds low for as long as SPE=1, so it can't frame transactions.

Q11 — PCLK 50 MHz, slave max 10 MHz?

BR = 010 → ÷8 → 6.25 MHz. SCK = f_PCLK / 2^(BR+1). ÷4 gives 12.5 MHz, which is over the limit, and only powers of two exist. Check which bus feeds the SPI: SPI1 runs on APB2 (up to 100 MHz), SPI2/SPI3 on APB1 (up to 50 MHz).

Q12 — Why configure CR1 with SPE=0?

Those CR1 bits drive the hardware directly. Flipping CPOL while enabled makes SCK jump; the slave counts that as an edge, so every following byte is shifted by one bit. Changing BR or DFF mid-frame corrupts the current transfer. To switch modes between two slaves: BSY=0 → SPE=0 → change CPOL/CPHA → SPE=1 → only then lower the next CS.

Traps I fell into

These are the corrections from the Q&A. Check them first during review.

- **UART is full-duplex** (separate TX and RX wires). Only single-wire mode is half-duplex.
- **UART has no master and target.** The two sides are equal peers.
- **SPI's clock is SCK.** SCL is I2C's name.
- **I2C needs addresses because its wires are shared.** Open-drain is what enables ACK, clock stretching and arbitration.
- **I2C pulled-up lines are not floating**, just weakly driven high, which is why they rise slowly.
- **CPHA means first or second edge**, not rising or falling. Name the actual edge when answering.
- **Wait for BSY=0 before raising CS** to protect the current slave's last byte, not to avoid two slaves talking at once.
- **SSOE=1 doesn't cause MODF.** The dangerous case is the reset default SSM=0, SSOE=0.

- **SSM=1 alone still faults.** Set SSI=1 as well.
- **Pin cost is 3 + N**, not N.

Still to do

- [ ] In a few days: answer Q1–12 out loud without notes (10 minutes); reread wherever you hesitate
- [ ] After the RTOS mutex topic: Q19 (ISR and main() sharing one SPI bus)
- [ ] Next protocol break: Q13–20 (M95320 WREN and page wrap, MCP2515 BIT MODIFY and stuck INT, 0xFF debugging, verifying without a logic analyzer, slow-SCK fixes)
- [ ] Open follow-ups: why a 100 Ω I2C pull-up is too small · which byte each slave latches in the daisy chain (0xAA, 0xBB, 0xCC) · UART ±5% vs ±1% baud mismatch · what the CR1 write must do after clearing MODF